

Article

Particle Swarm Optimization Algorithm Using Velocity Pausing and Adaptive Strategy

Kezong Tang and Chengjian Meng *

School of Information Engineering, Jingdezhen Ceramic University, Jingdezhen 333403, China;
tangkezong@jci.edu.cn

* Correspondence: 2220045004@stu.jcu.edu.cn

Abstract: Particle swarm optimization (PSO) as a swarm intelligence-based optimization algorithm has been widely applied to solve various real-world optimization problems. However, traditional PSO algorithms encounter issues such as premature convergence and an imbalance between global exploration and local exploitation capabilities when dealing with complex optimization tasks. To address these shortcomings, an enhanced PSO algorithm incorporating velocity pausing and adaptive strategies is proposed. By leveraging the search characteristics of velocity pausing and the terminal replacement mechanism, the problem of premature convergence inherent in standard PSO algorithms is mitigated. The algorithm further refines and controls the search space of the particle swarm through time-varying inertia coefficients, symmetric cooperative swarms concepts, and adaptive strategies, balancing global search and local exploitation. The performance of VASPSO was validated on 29 standard functions from Cec2017, comparing it against five PSO variants and seven swarm intelligence algorithms. Experimental results demonstrate that VASPSO exhibits considerable competitiveness when compared with 12 algorithms. The relevant code can be found on our project homepage.

Keywords: particle swarm optimization; adaptive strategy; velocity pausing; terminal replacement mechanism; symmetric cooperative swarms



Citation: Tang, K.; Meng, C. Particle Swarm Optimization Algorithm Using Velocity Pausing and Adaptive Strategy. *Symmetry* **2024**, *16*, 661.
<https://doi.org/10.3390/sym16060661>

Academic Editors: Lei Yue and Jinhua Zhang

Received: 20 April 2024

Revised: 15 May 2024

Accepted: 16 May 2024

Published: 27 May 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Swarm intelligence algorithms are computational methods inspired by collective behaviors observed in nature. They simulate interactions and cooperation among individuals within a group to solve complex problems. The development of swarm intelligence algorithms stems from recognizing the limitations of traditional, individual-based algorithms in tackling complex challenges. Conventional algorithms often rely on deterministic, single-agent methods, which may struggle to find global optima or encounter significant computational complexity for certain problems [1,2]. To address these challenges, researchers have introduced a new algorithmic paradigm: swarm intelligence algorithms. These algorithms collectively solve problems by simulating information exchange, cooperation, and competition among group members. Notable examples of swarm intelligence algorithms include genetic algorithms, ant colony algorithms, and bee swarm algorithms, which have demonstrated significant success across various fields.

Particle swarm optimization (PSO), initially proposed by Eberhart and Kennedy in 1995 [3], is inspired by the behavior of bird flocks and fish schools. In PSO, solutions to problems are represented by a swarm of particles, with each particle embodying a solution, and the swarm collectively representing a potential set of solutions within the solution space. Particles optimize their positions and velocities by continuously searching and iterating within the solution space, learning from and memorizing the best solutions found. PSO offers several advantages: it has a rapid convergence rate and robust global search capabilities; its algorithmic structure is simple, facilitating easy implementation and

application; and it adapts well to the continuity and non-linearity of various problems. Consequently, PSO has been extensively applied in domains such as function optimization, combinatorial optimization [4], machine learning [5], and image processing, among others.

Since its introduction, PSO has evolved through years of research and development, leading to numerous improvements. Some notable enhancements include the following:

1. Modifications to parameters, such as constant and random inertia weights [6,7], time-varying inertia [8–10], quadratic inertia weights [11], adaptive inertia weights [12,13], and sine chaos inertia weights [14] to enhance PSO's performance. Improvements to acceleration coefficients include time-varying acceleration coefficients [15], acceleration update strategies based on the Sigmoid function [16,17], and the addition of Gaussian white noise to acceleration coefficients [18] to balance the trade-off between individual experience and group collaboration;
2. Enhancements in the topological structure, which concerns how individuals within the algorithm communicate with each other. Proposals include selecting the nearest particles as neighbors in each generation [19], recombining randomly divided sub-populations after evolving for certain generations [20], and optimizing algorithms using ring topological structures. The proposed HIDMS-PSO variant enhances the traditional fixed topology of the unit structure with new master- and slave-dominated topologies [21], a random triplet topological structure that allows each particle to communicate with two random particles in the group based on its best position [22], and the particle swarm optimization algorithm with an extra gradient method (EPSO) [23] have been introduced to increase algorithmic diversity;
3. Improvements to learning strategies include the integration of four particle swarm search strategies into a strategy dynamics-based optimization method [24], a particle swarm optimization with multi-dimensional learning strategy based on adaptive inter-weight [25], and an adaptive hierarchical update particle swarm optimization algorithm employing a multi-choice comprehensive learning strategy comprising weighted composite sub-strategies and average evolutionary sub-strategies. Enhanced learning strategies and crossover operators (PSOLC) [26], and a multi-objective particle swarm with alternate learning strategies (MOPSOALS) [27], utilizing adaptive parameter updating exploratory and exploitative role learning strategies to improve optimization performance, have been proposed. To address large-scale variable problems, a multi-strategy learning particle swarm optimization algorithm (MSL-PSO) [28] employing different learning strategies at various stages has been introduced. To tackle deficiencies in solving complex optimization problems, a hybrid particle swarm optimization algorithm based on an adaptive strategy (ASPSO) [29] has been put forward.
4. Hybrid research involving the integration of other algorithms with PSO to better address complex optimization problems includes combining genetic factors with PSO, such as selection operators [30], crossover operators [31,32], and mutation operators [33]. Similarly, merging the simulated annealing algorithm to improve the global optimum update strategy in PSO [34], hybridizing with the grey wolf optimization algorithm [35], pattern search algorithm [36], crow search algorithm [37], hybrid particle filtering (PF) [38], and hybrid improved symbiotic organisms search (MSOS) [39] to fully leverage each algorithm's strengths.

Despite the maturity and successful application of these classic swarm intelligence algorithms in solving real-world optimization challenges, issues like premature convergence and suboptimal performance persist in complex application areas. Moreover, as societal demands evolve, real-world optimization problems increasingly exhibit high-dimensionality, multiple constraints, and multimodality, posing new challenges to classical optimization algorithms such as swarm intelligence algorithms. Against this backdrop, enhancing swarm intelligence algorithms to better tackle increasingly complex optimization problems has emerged as a research focal point.

To augment the particle swarm algorithm's capacity to solve complex optimization problems and to more effectively balance exploration and exploitation, this paper introduces

an improved particle swarm algorithm dubbed VASPSO. VASPSO contributes in five ways as described below:

- Introducing a time-varying inertia coefficient to enhance the algorithm's convergence speed and global search capability;
- Adopting velocity pausing concepts to mitigate premature convergence issues in particle swarm algorithms;
- Employing an adaptive strategy to dynamically fine-tune the balance between exploration and exploitation, thus boosting algorithm performance;
- Incorporating symmetric cooperative swarms concepts, treating different particles with varied strategies to aid the algorithm in discovering optimal solutions;
- Implementing a terminal replacement mechanism to excise the least performing particles, thereby refining the accuracy of the final solution;

Moreover, to ascertain the viability of the newly proposed VASPSO algorithm, it was tested against 29 benchmark functions from CEC2017, comparing its performance with five PSO variants and seven other swarm intelligence algorithms. Experimental outcomes reveal VASPSO's superiority over other algorithms in most benchmark functions, underscoring its success.

The remainder of this paper is structured as follows: Section 2 elucidates the standard particle swarm algorithm. The VASPSO algorithm is expounded in Section 3, with Section 4 showcasing the test functions and outcomes of experimental simulations. Conclusions and directions for future work are presented in Section 5.

2. Standard Particle Swarm Optimization

Particle swarm optimization (PSO) is a swarm intelligence-based optimization algorithm inspired by the foraging behavior of bird flocks [3]. The core principle of PSO is to optimize a search process by simulating the behavior of particles within a swarm. Each particle in the swarm represents a candidate solution, navigating the search space by updating its position and velocity based on both its own experience and the collective information from the swarm. Specifically, the position of a particle corresponds to a candidate solution's value, while its velocity determines the direction and magnitude of the search. Initially, particles are dispersed randomly throughout the search space with varying velocities. Their performance is assessed using a fitness function, which informs the updates to both the best position discovered by each particle (personal best) and the best overall position found by the swarm (global best). Throughout the algorithm, particles dynamically adjust their velocity and position, leveraging insights from their personal best achievements and the global best discoveries, in pursuit of optimal solutions. Below is the initialization of particles in the particle swarm optimization algorithm:

$$\mathbf{X}^0 = [X_{i1}^0, X_{i2}^0, \dots, X_{iD}^0], \quad i = 1, 2, \dots, N \quad (1)$$

$$\mathbf{V}^0 = [V_{i1}^0, V_{i2}^0, \dots, V_{iD}^0], \quad i = 1, 2, \dots, N \quad (2)$$

The w in Formula (4) is called the inertia weight and is derived from Formula (3):

$$\omega^{t+1} = \omega^t - \frac{(\omega^t - \omega^{end}) \cdot (t - t_{start})}{t_{end} - t_{start}} \quad (3)$$

The formula for updating the velocity and position of the particle is as follows:

$$\begin{aligned} V(t+1) &= w \cdot V(t) + c1 \cdot \text{rand} \cdot (P_{\text{best}} - X(t)) \\ &\quad + c2 \cdot \text{rand} \cdot (G_{\text{best}} - X(t)) \end{aligned} \quad (4)$$

$$X(t+1) = X(t) + V(t+1) \quad (5)$$

where c_1 and c_2 are called acceleration coefficients, $V(t)$ represents the velocity of the particle at time t , $X(t)$ represents the position of the particle at time t , P_{best} represents the individual best position of the particle, and G_{best} represents the global best position of the group.

3. Particle Swarm Optimization Algorithm Using Velocity Pausing and Adaptive Strategy

The proposed approach aims to mitigate premature convergence and enhance the optimization performance of particle swarm optimization (PSO) by integrating velocity pausing and adaptive strategy. In VASPSO, five key enhancements are elaborated below.

3.1. Time-Varying Inertia Weight

The modification of the inertia weight is a critical aspect of the particle swarm optimization (PSO) algorithm that significantly enhances its efficacy. Initially, the introduction of velocity pausing aimed to augment the global search capabilities of PSO and to mitigate premature convergence to local optima. However, while this strategy can enhance exploration, it may also, over time, lead to a slower convergence rate and, in some instances, convergence challenges. To address these challenges, particularly in the later stages of execution, a time-decreasing weight function is often implemented to adjust the inertia weight. This function is designed to progressively reduce the inertia weight across iterations, maintaining robust global search capabilities while simultaneously facilitating quicker convergence towards the optimal solution. By dynamically modulating the inertia weight, PSO strikes an improved balance between exploration and exploitation, thereby optimizing both the convergence speed and the algorithm's overall global search performance.

The inertia weight is calculated as follows:

$$w(t) = \exp^{-\left(\frac{bt}{T}\right)^b} \quad (6)$$

where T is the maximum number of iterations and b is the set constant.

3.2. Velocity Pausing

In 2023, Tareq M. Shami introduced the concept of velocity pausing into the particle swarm optimization (PSO) algorithm, marking a significant advancement in its performance [40]. This modification allows each particle the option to temporarily suspend velocity updates during iterations, instead selecting from three predefined velocities: slow, fast, and constant. This addition of a third velocity option diverges from traditional PSO mechanics and increases the algorithm's flexibility, promoting a more balanced approach between exploration and exploitation. Such enhancements address prevalent challenges in classical PSO algorithms, including premature convergence and the tendency to become trapped in local optima.

The careful selection of the pausing coefficient is critical. When the pausing coefficient approaches 1, the particle's velocity update closely mirrors that of the classical PSO. Conversely, when the coefficient is set very low, it forces the particle's velocity to remain constant, potentially leading to slower convergence rates and difficulties in discovering superior solutions.

The formula for velocity pausing is as follows:

$$V_i(t+1) = \begin{cases} V_i(t) & \text{if } rand < a \\ w(t)V_i(t) & \text{Otherwise} \\ +c_1r_1(P_{\text{best}_i}(t) - X_i(t)) & \\ +c_2r_2(G_{\text{best}_i}(t) - X_i(t)) & \end{cases} \quad (7)$$

where a is the coefficient of velocity pausing, and c_1 and c_2 are the learning factors.

The particle position update formula is as follows:

$$X_i(t+1) = \begin{cases} \text{Gbest} + wr_3 \mid \text{Gbest} \mid^{w(t)} & \text{if } rand < 0.49 \\ \text{Gbest} - wr_4 \mid \text{Gbest} \mid^{w(t)} & \text{Otherwise} \end{cases} \quad (8)$$

3.3. Adaptive Strategy

Traditional algorithms that employ a single-strategy approach for position updates often struggle to achieve a suitable balance between exploration and exploitation, particularly in complex scenarios characterized by multiple local optima or highly multimodal objective functions. In contrast, adaptive strategy position updates can significantly enhance the algorithm's search efficiency. This approach involves dynamically optimizing the search space that may yield the best solution by adaptively selecting the position update strategy during each iteration. Such improvements have proven to markedly increase search efficiency and have been widely adopted in enhancing both traditional and classical algorithms [29].

The core of the adaptive strategy for position updates resides in the iterative modification of the parameter p , which is adjusted based on changes in the fitness value. This responsive adjustment enables the algorithm to better navigate the solution landscape, effectively balancing the dual imperatives of exploring new possibilities and exploiting known good solutions.

The formula for parameter p is as follows:

$$p_i = \frac{\exp(\text{fit}(X_i(t)))}{\exp\left(\frac{1}{N} \sum_{i=1}^N \text{fit}(X_i(t))\right)} \quad (9)$$

where N is the total population. $\text{fit}()$ is the fitness of the corresponding particle, and p_i is the ratio of the current fitness of the particle to the average fitness of the population. An estimate is obtained in each iteration.

The location update formula is as follows:

$$X_i(t+1) = \begin{cases} \omega(t)X_i(t) + (1 - \omega(t))V_i(t+1) \\ + \text{Gbest}(t) & p_i > rand \\ X_i(t) + V_i(t+1) & \text{otherwise} \end{cases} \quad (10)$$

When p_i is small, the adaptation value of particle i exceeds the average level of the population. In this scenario, to augment the global exploration capabilities of particle i , the location update strategy $X = X + V$ is employed. Conversely, when p_i is large, indicating that particle i is performing below the average level of the population, the location update strategy $X = \omega X + (1 - \omega)V$ is implemented to boost its local exploitation capabilities.

3.4. Symmetric Cooperative Swarms

Symmetry is a prominent characteristic in algorithms and their applications, playing a crucial role across various domains. In algorithms, symmetry often reveals patterns within data, aiding in simplifying complex problems and enhancing algorithm efficiency. One standout example is the divide-and-conquer approach. By leveraging this principle, the search space is decomposed for exploration, thereby reducing time complexity. In image processing, symmetry is frequently utilized to optimize rendering and image manipulation algorithms. For instance, in 3D graphics, objects typically exhibit symmetrical properties, allowing for reduced computational overhead in rendering and processing, thus improving graphics processing efficiency. In database optimization, exploiting data symmetry can reduce redundant storage and enhance query efficiency. Techniques such as normalization and denormalization are employed to eliminate duplicate and redundant information from data, while indexing and caching techniques accelerate data access and query operations.

In this paper, we also draw inspiration from the concept of symmetry. During each iteration, the entire particle swarm is divided based on the fitness of particles, segregating

them into superior and inferior particle clusters. This symmetrical partitioning enables particles to undertake different responsibilities and tasks, facilitating better exploration of the solution space.

The elite group implements advanced strategies to finely balance exploration and exploitation. Initially, a velocity pausing strategy is applied, temporarily halting the velocity updates under specific conditions to allow prolonged investigation within local solution spaces, thereby enhancing the exploration of potentially superior solutions. Additionally, an adaptive strategy modifies the positional behavior based on the problem's characteristics and the group's current state, dynamically adjusting the balance between exploration and exploitation to suit the complexity and diversity of the problem.

Conversely, the non-elite group primarily updates positions by referencing the global best solution, fostering broader exploration of the solution space. This group's focus is on uncovering new potential solutions, thereby injecting diversity into the swarm. By orienting their position updates solely around the global best, the non-elite particles can explore more aggressively, expanding the search range for the algorithm.

Through this cooperative strategy, the particle swarm optimization algorithm achieves a more effective balance between exploration and exploitation. The elite group conducts targeted exploration and refined exploitation using sophisticated strategies, such as velocity pausing and adaptive positional updates. In contrast, the non-elite group pursues extensive exploration driven by the global best solution. This structured approach allows the algorithm to search more comprehensively for viable solutions, thereby enhancing both the performance and the efficacy of the algorithm.

A detailed flowchart of the subgroup collaborative process is shown in Figure 1: Subgroup Collaborative Process Flowchart.

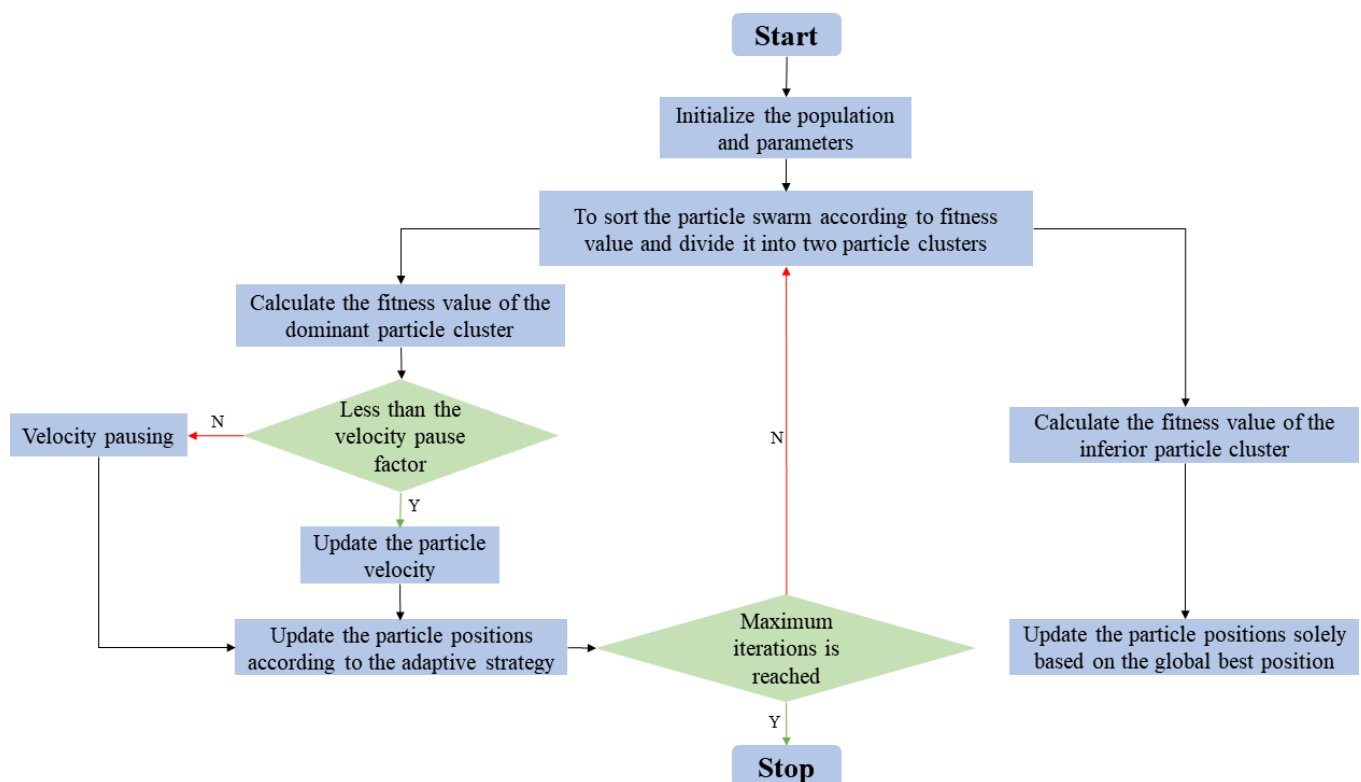


Figure 1. Flowchart of the VASPSO algorithm.

3.5. Terminal Replacement Mechanism

The terminal replacement mechanism, also known as the ‘last-place elimination mechanism,’ is a widely used algorithmic method for selecting optimal solutions under conditions of limited resources or intense competition. Inspired by the biological principle of natural selection, exemplified by the concept of survival of the fittest, this mechanism is prevalent in various optimization and genetic algorithms, aiding in the efficient identification of superior solutions.

Within this mechanism, an initial set of candidate solutions is generated and subsequently ranked according to predefined evaluation criteria. Based on these rankings, candidates with the lowest scores are progressively eliminated, while those with higher scores are retained for additional evaluation rounds. This iterative process is repeated until the optimal solution is identified or a predetermined stopping condition is reached.

To better mimic the unpredictability of real-world scenarios and enhance the algorithm’s robustness, the concept of an elimination factor has been incorporated into the particle swarm optimization (PSO) algorithm. This modification involves selectively replacing particles according to specific rules, with a predetermined probability that permits some particles to avoid elimination due to various factors. If a randomly generated number falls below the established elimination factor, a replacement operation is executed; otherwise, the particle is preserved for further evaluation. By integrating this elimination factor, the algorithm not only replicates the inherent uncertainties of real-life situations but also significantly enhances its adaptability and robustness.

$$Gbad(t) = \operatorname{argmax}\{\operatorname{fit}(Pbest_1(t)), \dots, \operatorname{fit}(Pbest_N(t))\} \quad (11)$$

Among them, $Gbad$ is the least fit particle among all the individual optimal particles.

$$Nbest(t) = Gbest(t) + \operatorname{rand} \cdot (Pbest_h(t) - Pbest_o(t)), \quad h \neq o \in \{1, 2, \dots, N\} \quad (12)$$

$Nbest$ is generated by the intersection of the global best particle and any two individual best particles.

$$Gbad(t) = \begin{cases} Nbest(t) & \operatorname{fit}(Nbest(t)) < \operatorname{fit}(Gbad(t)) \\ Gbad(t) & \text{otherwise} \end{cases} \quad (13)$$

The fitness values of the new cross-generated particle $Nbest$ and $Gbad$ are compared. If $Nbest$ is better than $Gbad$, $Gbad$ is replaced by $Nbest$; otherwise, $Gbad$ remains unchanged, and the eliminated particle is selected.

In summary, VASPSO is proposed as a fusion variant, and its pseudo-code is shown in Algorithm 1.

Algorithm 1 VASPSO Algorithm.

```

1: Initialize parameters, population size  $N$ , maximum iterations  $T$ 
2: for  $t = 1$  to  $T$  do
3:   Calculate the fitness of each particle
4:   Sort all particles and divide them into two groups: elite and non-elite based on
   fitness values
5:   For the first elite group
6:   for  $i = 1$  to  $HN$  do
7:     if random number  $< r_1$  then
8:       Update velocity according to velocity pausing Formula (7)
9:     end if
10:    Update position according to the adaptive strategy Formula (10)
11:  end for
12:  For the second non-elite group
13:  for  $i = HN + 1$  to  $N$  do
14:    Update position according to velocity pausing Formula (8)
15:  end for
16:  % Last-place elimination mechanism
17:  for  $i = 1$  to  $N$  do
18:    Update  $G_{bad}$  according to Formula (11)
19:    Update  $N_{best}$  according to Formula (12)
20:    if rand  $< 0.99$  then
21:      if  $fit(N_{best}) < fit(G_{bad})$  then
22:         $G_{bad} \leftarrow N_{best}$ 
23:         $fit(G_{bad}) \leftarrow fit(N_{best})$ 
24:      end if
25:    end if
26:  end for
27:  Evaluate fitness and update personal best and global best solutions.
28:  for  $i = 1$  to  $N$  do
29:    if  $fX(i) < fP_{best}(i)$  then
30:       $P_{best}(i, :) \leftarrow X(i, :)$ 
31:       $fP_{best}(i) \leftarrow fX(i)$ 
32:    end if
33:    if  $fP_{best}(i) < g_{bestValue}$  then
34:       $G_{best} \leftarrow P_{best}(i, :)$ 
35:       $g_{bestValue} \leftarrow fP_{best}(i)$ 
36:    end if
37:  end for
38: end for
39: Return  $g_{bestValue}$ 

```

4. Experimental Results and Discussion**4.1. Test the Function and Compare the Algorithm**

In this experiment, 29 benchmark functions from CEC2017 were selected to evaluate the performance of the proposed MPSO algorithm (f2 was excluded due to its unstable behavior at higher dimensions and significant performance variations in the same algorithm implemented in Matlab). These benchmark functions are divided into four categories: f1–f3 are unimodal functions, f4–f10 are multimodal functions, f11–f20 are hybrid functions, and f21–f30 are composition functions [41]. These functions are listed in Table 1.

To validate the performance of VASPSO, five PSO variants were employed: VPPSO [40], MPSO [42], OldMPSO [43] (to distinguish from the previous MPSO), AGPSO [44], and IPSO [45]; along with seven other swarm intelligence algorithms, including DE [46], GWO [47], CSO [48], DBO [49], BWO [50], SSA [51], and ABC [52]. These algorithms are listed in Table 2.

Table 1. The benchmark function used in this paper.

NO.	Function	D	Range	f_{opt}
f1	Shifted and Rotated Bent Cigar Function	30	[−100, 100]	100
f3	Shifted and Rotated Zakharov Function	30	[−100, 100]	300
f4	Shifted and Rotated Rosenbrock's Function	30	[−100, 100]	400
f5	Shifted and Rotated Rastrigin's Function	30	[−100, 100]	500
f6	Shifted and Rotated Expanded Scaffer's F6 Function	30	[−100, 100]	600
f7	Shifted and Rotated Lunacek Bi-Rastrigin Function	30	[−100, 100]	700
f8	Shifted and Rotated Non-Continuous Rastrigin's Function	30	[−100, 100]	800
f9	Shifted and Rotated Levy Function	30	[−100, 100]	900
f10	Shifted and Rotated Schwefel's Function	30	[−100, 100]	1000
f11	Hybrid Function 1 (N = 3)	30	[−100, 100]	1100
f12	Hybrid Function 2 (N = 3)	30	[−100, 100]	1200
f13	Hybrid Function 3 (N = 3)	30	[−100, 100]	1300
f14	Hybrid Function 4 (N = 4)	30	[−100, 100]	1400
f15	Hybrid Function 5 (N = 4)	30	[−100, 100]	1500
f16	Hybrid Function 6 (N = 4)	30	[−100, 100]	1600
f17	Hybrid Function 6 (N = 5)	30	[−100, 100]	1700
f18	Hybrid Function 6 (N = 5)	30	[−100, 100]	1800
f19	Hybrid Function 6 (N = 5)	30	[−100, 100]	1900
f20	Hybrid Function 6 (N = 6)	30	[−100, 100]	2000
f21	Composition Function 1 (N = 3)	30	[−100, 100]	2100
f22	Composition Function 2 (N = 3)	30	[−100, 100]	2200
f23	Composition Function 3 (N = 4)	30	[−100, 100]	2300
f24	Composition Function 4 (N = 4)	30	[−100, 100]	2400
f25	Composition Function 5 (N = 5)	30	[−100, 100]	2500
f26	Composition Function 6 (N = 5)	30	[−100, 100]	2600
f27	Composition Function 7 (N = 6)	30	[−100, 100]	2700
f28	Composition Function 8 (N = 6)	30	[−100, 100]	2800
f29	Composition Function 9 (N = 3)	30	[−100, 100]	2900
f30	Composition Function 10 (N = 3)	30	[−100, 100]	3000

Table 2. Algorithm parameter information.

Algorithm	Year	Parameter Information
VPPSO	2023	$w = 0.1 \sim 0.9$, $\alpha = 0.3$, $N_1 = 50$, $N_2 = 50$, $c_1 = c_2 = 2$
MPSO	2020	$w = 0.1 \sim 0.9$, $c_1 = c_2 = 2$
OldMPSO	2009	$w = 0.4 \sim 0.9$, $c_1 = (-2.05/T)t + 2.55$, $c_2 = (1/T)t + 1.25$
AGPSO	2014	$w = 0.4 \sim 0.9$, $c_1 = c_2 = 2$
IPSO	2008	$c_1 = 2.5 + 2(t/T)^2 - 2(2t/T)$, $c_2 = 0.5 - 2(t/T)^2 + 2(2t/T)$
DE	1997	$F = 0.8$, $CR = 0.1$
GWO	2014	$SearchAgents = 50$
CSO	2014	$CDC = 0.8$, $SMP = 5$, $c = 2.05$
DBO	2022	$percent = 0.2$
BWO	2022	$B_f = 0.5 \sim 1$, $B_f < W_f$
SSA	2020	$Params.nVar = 2$
ABC	2007	$limit = 100$, $Size\ of\ employed\ bee = N/2$
VASPSO	Presented	

To be fair, each algorithm is run 50 times independently, and the termination condition for all algorithms is the maximum number of iterations, which is set to 600.

4.2. Comparison between VASPSO and 12 Algorithms

In this section, VASPSO is compared with five variant PSOs and seven other swarm intelligence algorithms, with the results listed in Table 2 through 7, respectively. Three quantitative metrics are used: mean (Mean), standard deviation (Std), and minimum value (Min).

The convergence curves of the algorithms are shown in Figures 2 and 3. The trajectory curves represent the search process of the particles in each dimension. The fluctuation of the former indicates that the particles are conducting a global search, while the stability of the latter suggests that the particles have reached a global or local optimum.

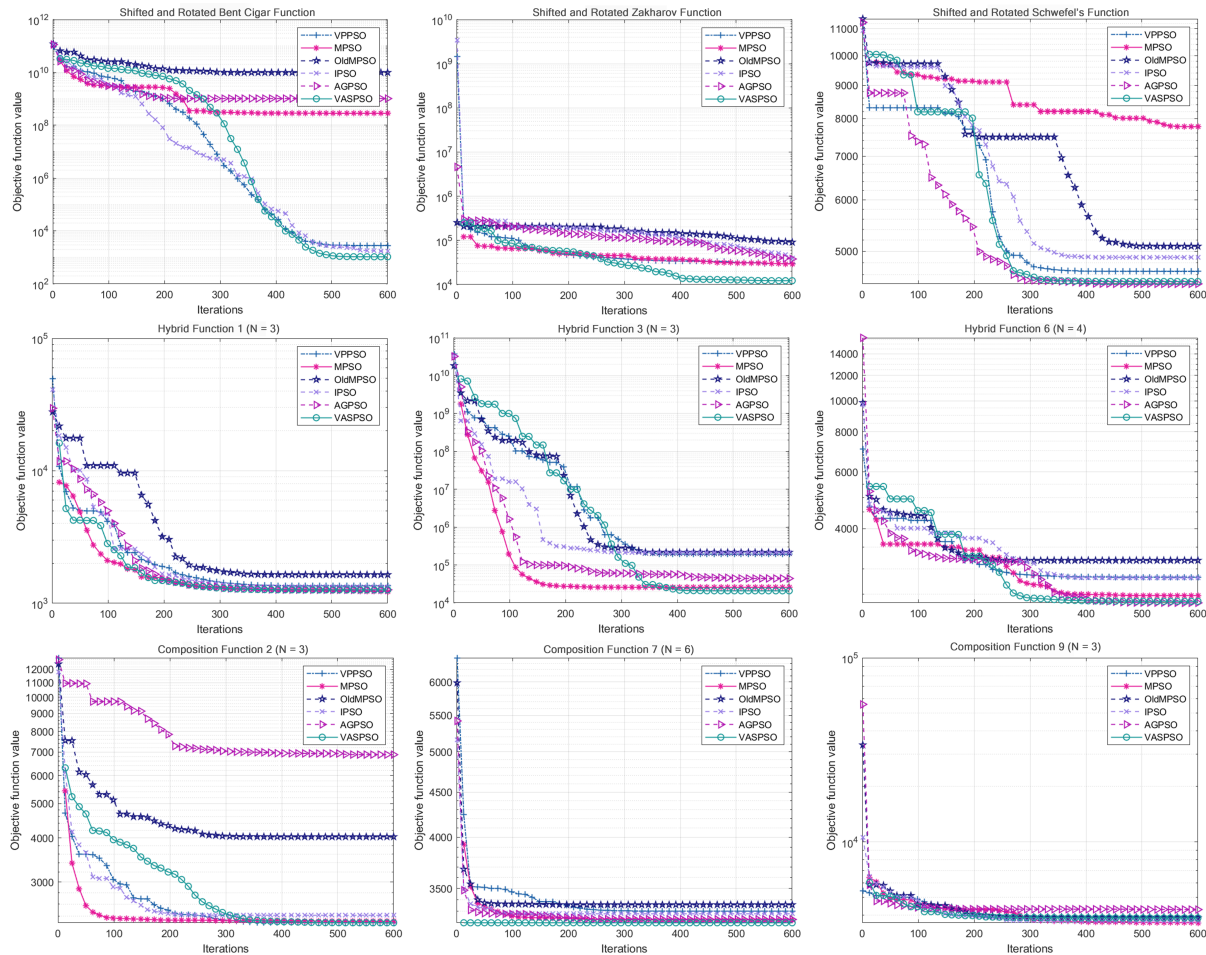


Figure 2. Convergence curve of algorithms for variants of PSO.

From Table 3, it is evident that VASPSO generally outperforms the other five PSO variants. In terms of mean values, VASPSO shows superior performance on functions f_3 , f_{10} , f_{11} , and f_{27} compared to the other variants. In terms of variance, VASPSO demonstrates greater stability on functions f_3 , f_5 , f_8 , f_9 , f_{11} , f_{16} , f_{21} , f_{25} , and f_{27} than the other variants. In terms of minimum values, VASPSO excels on functions f_3 , f_4 , f_{10} , f_{11} , f_{14} , f_{18} , f_{22} , f_{25} , f_{27} , f_{28} , and f_{29} compared to the other variants. Although VASPSO does not perform well in some cases, it still ranks as the best in performance when considering the average ranking across all functions. The convergence of the algorithm is shown in Figure 2.

From Table 4, it is clear that VASPSO overall surpasses the other seven swarm intelligence algorithms, with superior performance on functions f_3 , f_{11} , f_{13} , f_{14} , f_{15} , f_{19} , and f_{28} compared to the other algorithms. In terms of variance, VASPSO shows greater stability on functions f_{11} , f_{13} , f_{14} , and f_{15} than the other swarm intelligence algorithms. In terms of minimum values, VASPSO performs better on functions f_1 , f_3 , f_4 , f_{10} , f_{11} , f_{13} , f_{14} , f_{18} , f_{22} , f_{25} , f_{26} , f_{28} , f_{29} , and f_{30} compared to the other algorithms. Despite some poor performances on specific functions, VASPSO still emerges as the best in the overall ranking. The convergence of the algorithm is shown in Figure 3.

Table 3. Comparisons of experimental results between 7 evolutionary algorithms.

		DE	GWO	CSO	DBO	BWO	SSA	ABC	VASPSO
f1	Mean	7.29×10^6	1.72×10^9	1.71×10^9	3.97×10^7	6.73×10^{10}	5.80×10^{10}	2.82×10^8	1.12×10^8
	Std	1.72×10^6	1.64×10^9	1.21×10^9	3.51×10^7	6.02×10^9	8.54×10^9	1.34×10^8	5.31×10^8
	Min	4.50×10^6	4.79×10^7	1.27×10^8	6.41×10^4	4.78×10^{10}	3.47×10^{10}	5.55×10^7	1.04×10^3
	Rank	1	5	6	2	8	7	4	3
f3	Mean	1.15×10^5	5.19×10^4	2.91×10^4	8.69×10^4	9.32×10^4	8.39×10^4	3.42×10^5	1.39×10^4
	Std	1.95×10^4	1.28×10^4	8.54×10^3	2.28×10^4	8.56×10^3	6.05×10^3	1.05×10^5	7.13×10^3
	Min	5.97×10^4	2.29×10^4	1.53×10^4	4.89×10^4	7.54×10^4	6.61×10^4	1.91×10^5	4.25×10^3
	Rank	7	4	2	5	6	3	8	1
f4	Mean	6.04×10^2	5.89×10^2	5.73×10^2	6.08×10^2	1.78×10^4	1.60×10^4	6.74×10^2	6.10×10^2
	Std	1.72×10^1	6.10×10^1	4.09×10^1	8.16×10^1	1.91×10^3	2.72×10^3	5.58×10^1	1.27×10^2
	Min	5.59×10^2	5.18×10^2	4.94×10^2	4.96×10^2	1.28×10^4	1.03×10^4	5.39×10^2	4.60×10^2
	Rank	3	2	1	5	8	7	6	4
f5	Mean	6.74×10^2	6.21×10^2	6.86×10^2	7.27×10^2	9.44×10^2	8.92×10^2	7.48×10^2	6.89×10^2
	Std	1.05×10^1	4.12×10^1	3.54×10^1	4.60×10^1	1.65×10^1	3.38×10^1	1.73×10^1	2.43×10^1
	Min	6.43×10^2	5.59×10^2	6.21×10^2	6.52×10^2	9.07×10^2	7.42×10^2	7.11×10^2	5.97×10^2
	Rank	1	2	4	7	6	8	5	3
f6	Mean	6.02×10^2	6.09×10^2	6.33×10^2	6.35×10^2	6.93×10^2	6.92×10^2	6.13×10^2	6.51×10^2
	Std	2.52×10^{-1}	4.03×10^0	1.11×10^1	1.01×10^1	3.68×10^0	6.03×10^0	2.77×10^0	6.06×10^0
	Min	6.01×10^2	6.02×10^2	6.14×10^2	6.13×10^2	6.84×10^2	6.81×10^2	6.08×10^2	6.29×10^2
	Rank	1	3	5	4	7	8	2	6
f7	Mean	9.26×10^2	8.64×10^2	9.84×10^2	9.68×10^2	1.47×10^3	1.41×10^3	9.89×10^2	1.10×10^3
	Std	1.19×10^1	3.29×10^1	4.95×10^1	9.68×10^1	2.93×10^1	4.71×10^1	1.54×10^1	6.26×10^1
	Min	9.03×10^2	8.00×10^2	8.85×10^2	8.14×10^2	1.37×10^3	1.26×10^3	9.46×10^2	8.65×10^2
	Rank	2	1	5	4	8	7	3	6
f8	Mean	9.75×10^2	8.95×10^2	9.74×10^2	1.01×10^3	1.15×10^3	1.09×10^3	1.05×10^3	9.38×10^2
	Std	1.25×10^1	3.25×10^1	2.89×10^1	4.83×10^1	1.44×10^1	2.91×10^1	1.37×10^1	2.25×10^1
	Min	9.29×10^2	8.46×10^2	9.19×10^2	9.33×10^2	1.11×10^3	9.90×10^2	1.01×10^3	8.73×10^2
	Rank	3	2	4	6	7	8	5	1
f9	Mean	2.89×10^3	2.01×10^3	4.74×10^3	5.05×10^3	1.10×10^4	1.01×10^4	5.94×10^3	4.43×10^3
	Std	4.42×10^2	6.33×10^2	1.50×10^3	1.79×10^3	7.67×10^2	1.21×10^3	1.22×10^3	4.86×10^2
	Min	1.99×10^3	1.03×10^3	2.01×10^3	1.79×10^3	8.63×10^3	6.97×10^3	3.86×10^3	2.46×10^3
	Rank	2	1	5	4	8	7	6	3
f10	Mean	6.61×10^3	4.87×10^3	5.74×10^3	5.71×10^3	8.50×10^3	8.38×10^3	9.15×10^3	4.89×10^3
	Std	3.52×10^2	1.51×10^3	4.77×10^2	8.54×10^2	2.88×10^2	6.41×10^2	3.20×10^2	4.84×10^2
	Min	5.79×10^3	3.09×10^3	4.29×10^3	3.94×10^3	7.71×10^3	7.02×10^3	7.82×10^3	2.97×10^3
	Rank	5	2	3	4	6	7	8	1
f11	Mean	2.01×10^3	2.10×10^3	1.59×10^3	1.62×10^3	1.90×10^4	8.16×10^3	1.13×10^4	1.27×10^3
	Std	4.77×10^2	9.88×10^2	2.31×10^2	2.55×10^2	7.55×10^3	1.84×10^3	2.35×10^3	5.36×10^1
	Min	1.34×10^3	1.30×10^3	1.28×10^3	1.27×10^3	8.14×10^3	4.40×10^3	6.00×10^3	1.15×10^3
	Rank	4	5	2	3	8	6	7	1
f12	Mean	2.02×10^7	8.41×10^7	5.67×10^7	4.99×10^7	1.68×10^{10}	1.28×10^{10}	4.91×10^8	6.38×10^7
	Std	6.78×10^6	8.86×10^7	4.23×10^7	8.66×10^7	3.11×10^9	2.97×10^9	1.59×10^8	7.73×10^7
	Min	8.10×10^6	3.30×10^6	5.73×10^6	2.27×10^5	9.52×10^9	5.85×10^9	1.72×10^8	2.57×10^5
	Rank	2	5	4	1	8	7	6	3
f13	Mean	3.52×10^6	2.55×10^7	2.48×10^6	4.32×10^6	1.31×10^{10}	8.23×10^9	1.03×10^7	1.08×10^5
	Std	1.42×10^6	9.05×10^7	8.46×10^6	1.22×10^7	4.78×10^9	5.07×10^9	1.09×10^7	6.85×10^4
	Min	1.21×10^6	3.26×10^4	2.64×10^4	2.88×10^4	3.31×10^9	1.72×10^9	1.05×10^6	2.14×10^4
	Rank	3	6	2	4	8	7	5	1
f14	Mean	2.58×10^5	4.92×10^5	8.55×10^4	1.03×10^5	1.48×10^7	4.33×10^6	3.39×10^5	2.97×10^4
	Std	1.27×10^5	6.16×10^5	8.60×10^4	1.03×10^5	1.09×10^7	4.63×10^6	1.61×10^5	3.01×10^4
	Min	3.69×10^4	4.26×10^3	4.21×10^3	4.93×10^3	6.96×10^5	1.60×10^5	1.07×10^5	1.61×10^3
	Rank	4	5	2	3	8	7	6	1
f15	Mean	7.23×10^5	1.85×10^6	5.23×10^5	7.78×10^4	1.20×10^9	7.96×10^8	1.57×10^6	3.30×10^4
	Std	3.98×10^5	9.18×10^6	7.73×10^5	6.46×10^4	6.72×10^8	5.09×10^8	8.86×10^5	1.89×10^4
	Min	1.39×10^5	8.40×10^3	1.41×10^4	4.75×10^3	9.23×10^7	2.60×10^7	3.66×10^5	5.54×10^3
	Rank	4	5	3	2	8	7	6	1
f16	Mean	2.82×10^3	2.54×10^3	2.76×10^3	3.19×10^3	7.50×10^3	6.17×10^3	3.97×10^3	2.96×10^3
	Std	1.77×10^2	3.83×10^2	2.85×10^2	3.77×10^2	1.38×10^3	1.26×10^3	1.77×10^2	2.69×10^2
	Min	2.37×10^3	2.05×10^3	2.05×10^3	2.32×10^3	5.23×10^3	4.06×10^3	3.63×10^3	2.10×10^3
	Rank	3	2	1	5	8	7	6	4

Table 3. Cont.

		DE	GWO	CSO	DBO	BWO	SSA	ABC	VASPSO
f17	Mean	2.07×10^3	2.05×10^3	2.08×10^3	2.51×10^3	7.50×10^3	4.86×10^3	2.98×10^3	2.34×10^3
	Std	1.00×10^2	1.48×10^2	1.69×10^2	2.51×10^2	6.17×10^3	2.20×10^3	1.12×10^2	2.36×10^2
	Min	1.89×10^3	1.79×10^3	1.85×10^3	1.83×10^3	3.33×10^3	2.94×10^3	2.71×10^3	1.81×10^3
	Rank	2	1	4	6	8	7	5	3
f18	Mean	1.15×10^6	1.47×10^6	5.67×10^5	1.54×10^6	2.23×10^8	6.57×10^7	1.61×10^7	8.19×10^5
	Std	4.98×10^5	2.45×10^6	3.48×10^5	3.66×10^6	1.43×10^8	5.34×10^7	8.62×10^6	1.33×10^6
	Min	2.82×10^5	9.05×10^4	3.45×10^4	1.19×10^5	1.20×10^7	7.48×10^5	1.60×10^6	1.22×10^4
	Rank	3	4	1	5	8	7	6	2
f19	Mean	6.67×10^5	1.98×10^6	1.72×10^6	1.05×10^6	1.11×10^9	9.88×10^8	1.35×10^5	8.60×10^4
	Std	3.72×10^5	5.22×10^6	1.31×10^6	1.66×10^6	4.71×10^8	5.32×10^8	1.42×10^5	2.59×10^5
	Min	1.03×10^5	8.09×10^3	3.59×10^4	2.02×10^3	3.19×10^8	2.09×10^8	6.80×10^3	2.29×10^3
	Rank	4	6	5	3	8	7	2	1
f20	Mean	2.39×10^3	2.43×10^3	2.53×10^3	2.65×10^3	2.96×10^3	3.10×10^3	3.07×10^3	2.63×10^3
	Std	9.14×10^1	1.58×10^2	1.13×10^2	2.03×10^2	1.16×10^2	1.70×10^2	1.24×10^2	1.70×10^2
	Min	2.20×10^3	2.19×10^3	2.36×10^3	2.29×10^3	2.70×10^3	2.64×10^3	2.85×10^3	2.19×10^3
	Rank	1	2	3	6	5	8	7	4
f21	Mean	2.47×10^3	2.39×10^3	2.43×10^3	2.53×10^3	2.70×10^3	2.74×10^3	2.56×10^3	2.47×10^3
	Std	1.27×10^1	3.01×10^1	4.54×10^1	5.00×10^1	6.64×10^1	5.95×10^1	1.03×10^1	2.85×10^1
	Min	2.44×10^3	2.35×10^3	2.23×10^3	2.40×10^3	2.51×10^3	2.54×10^3	2.53×10^3	2.37×10^3
	Rank	4	1	2	6	7	8	5	3
f22	Mean	3.27×10^3	4.69×10^3	2.60×10^3	5.52×10^3	9.65×10^3	9.46×10^3	1.04×10^4	5.26×10^3
	Std	2.29×10^2	1.75×10^3	1.39×10^2	2.17×10^3	6.17×10^2	7.20×10^2	3.91×10^2	1.45×10^3
	Min	2.87×10^3	2.43×10^3	2.36×10^3	2.33×10^3	7.77×10^3	7.36×10^3	9.39×10^3	2.30×10^3
	Rank	2	4	1	5	7	6	8	3
f23	Mean	2.82×10^3	2.78×10^3	2.79×10^3	2.92×10^3	3.86×10^3	3.51×10^3	2.92×10^3	3.00×10^3
	Std	1.24×10^1	4.86×10^1	4.02×10^1	7.55×10^1	1.78×10^2	1.40×10^2	1.33×10^1	7.18×10^1
	Min	2.78×10^3	2.69×10^3	2.73×10^3	2.80×10^3	3.37×10^3	3.22×10^3	2.89×10^3	2.78×10^3
	Rank	3	1	2	6	8	7	4	5
f24	Mean	3.04×10^3	2.96×10^3	2.94×10^3	3.10×10^3	4.19×10^3	3.69×10^3	3.09×10^3	3.20×10^3
	Std	1.42×10^1	6.61×10^1	3.88×10^1	6.92×10^1	3.44×10^2	1.70×10^2	1.57×10^1	5.92×10^1
	Min	3.01×10^3	2.87×10^3	2.87×10^3	3.00×10^3	3.52×10^3	3.41×10^3	3.06×10^3	2.96×10^3
	Rank	3	2	1	6	8	7	4	5
f25	Mean	2.96×10^3	2.99×10^3	3.01×10^3	2.95×10^3	5.74×10^3	4.93×10^3	2.98×10^3	2.95×10^3
	Std	1.50×10^1	3.88×10^1	4.91×10^1	5.37×10^1	4.08×10^2	4.43×10^2	2.07×10^1	2.06×10^1
	Min	2.93×10^3	2.93×10^3	2.93×10^3	2.89×10^3	4.46×10^3	4.02×10^3	2.94×10^3	2.88×10^3
	Rank	2	5	6	3	8	7	4	1
f26	Mean	5.14×10^3	4.79×10^3	4.22×10^3	6.47×10^3	1.26×10^4	1.17×10^4	5.84×10^3	6.34×10^3
	Std	3.98×10^2	3.98×10^2	8.62×10^2	9.35×10^2	1.05×10^3	1.11×10^3	1.46×10^2	1.03×10^3
	Min	4.32×10^3	4.16×10^3	3.10×10^3	3.35×10^3	8.73×10^3	8.05×10^3	5.43×10^3	2.80×10^3
	Rank	4	2	1	6	8	7	3	5
f27	Mean	3.26×10^3	3.25×10^3	3.32×10^3	3.29×10^3	4.68×10^3	4.36×10^3	3.20×10^3	3.20×10^3
	Std	6.20×10^0	1.94×10^1	8.06×10^1	6.31×10^1	3.84×10^2	3.78×10^2	6.21×10^{-5}	2.01×10^{-4}
	Min	3.25×10^3	3.22×10^3	3.23×10^3	3.24×10^3	3.96×10^3	3.74×10^3	3.20×10^3	3.20×10^3
	Rank	4	3	6	5	8	7	2	1
f28	Mean	3.35×10^3	3.42×10^3	3.40×10^3	3.46×10^3	8.28×10^3	7.31×10^3	3.30×10^3	3.32×10^3
	Std	1.79×10^1	8.84×10^1	1.32×10^2	4.01×10^2	5.15×10^2	7.18×10^2	4.99×10^{-1}	4.06×10^1
	Min	3.31×10^3	3.28×10^3	3.25×10^3	3.27×10^3	6.89×10^3	5.51×10^3	3.30×10^3	3.21×10^3
	Rank	4	5	3	6	8	7	2	1
f29	Mean	3.93×10^3	3.81×10^3	4.07×10^3	4.23×10^3	1.16×10^4	8.04×10^3	5.00×10^3	4.00×10^3
	Std	9.95×10^1	1.85×10^2	2.26×10^2	3.45×10^2	4.87×10^3	2.18×10^3	2.77×10^2	2.75×10^2
	Min	3.75×10^3	3.46×10^3	3.62×10^3	3.58×10^3	5.85×10^3	5.52×10^3	4.36×10^3	3.33×10^3
	Rank	3	1	4	5	8	7	6	2
f30	Mean	7.93×10^5	8.49×10^6	1.06×10^7	2.20×10^6	2.69×10^9	1.41×10^9	4.69×10^5	2.52×10^6
	Std	3.51×10^5	8.88×10^6	9.63×10^6	3.76×10^6	9.97×10^8	9.63×10^8	3.39×10^5	3.17×10^6
	Min	3.41×10^5	1.46×10^6	1.32×10^6	2.45×10^4	6.37×10^8	2.22×10^8	9.14×10^4	1.81×10^4
	Rank	3	5	6	4	8	7	1	2
	final rank	87	92	94	131	219	202	142	77

Table 4. Comparisons of experimental results between some well-known variants of PSO.

		VPPSO	MPSO	OldMPSO	AGPSO	IPSO	VASPSO
f1	Mean	7.53×10^3	9.77×10^7	7.71×10^9	1.38×10^9	9.63×10^8	1.12×10^8
	Std	1.83×10^4	1.41×10^8	5.26×10^9	2.06×10^9	1.66×10^9	5.31×10^8
	Min	1.00×10^2	3.10×10^3	3.69×10^3	4.54×10^2	6.85×10^2	1.04×10^3
	Rank	1	2	6	5	4	3
f3	Mean	3.78×10^4	2.47×10^4	7.10×10^4	3.41×10^4	4.03×10^4	1.39×10^4
	Std	1.02×10^4	9.11×10^3	2.71×10^4	1.74×10^4	1.96×10^4	7.13×10^3
	Min	1.60×10^4	5.88×10^3	3.02×10^4	1.33×10^4	1.25×10^4	4.25×10^3
	Rank	4	2	6	3	5	1
f4	Mean	5.09×10^2	3.49×10^4	1.20×10^3	5.95×10^2	6.12×10^2	6.10×10^2
	Std	2.04×10^1	8.53×10^3	5.97×10^2	1.93×10^2	2.32×10^2	1.27×10^2
	Min	4.72×10^2	1.84×10^4	5.26×10^2	4.90×10^2	4.75×10^2	4.60×10^2
	Rank	1	6	5	3	4	2
f5	Mean	6.58×10^2	1.11×10^3	6.43×10^2	6.06×10^2	6.14×10^2	6.89×10^2
	Std	4.06×10^1	5.76×10^1	4.09×10^1	2.75×10^1	3.50×10^1	2.43×10^1
	Min	5.81×10^2	1.00×10^3	5.57×10^2	5.58×10^2	5.65×10^2	5.97×10^2
	Rank	5	6	3	1	2	4
f6	Mean	6.39×10^2	7.24×10^2	6.20×10^2	6.09×10^2	6.12×10^2	6.51×10^2
	Std	9.14×10^0	1.21×10^1	8.36×10^0	4.80×10^0	6.77×10^0	6.06×10^0
	Min	6.21×10^2	6.82×10^2	6.08×10^2	6.03×10^2	6.04×10^2	6.29×10^2
	Rank	5	6	3	1	2	4
f7	Mean	9.50×10^2	2.55×10^3	9.06×10^2	8.64×10^2	9.10×10^2	1.10×10^3
	Std	8.71×10^1	2.73×10^2	7.67×10^1	5.24×10^1	5.41×10^1	6.26×10^1
	Min	8.19×10^2	1.91×10^3	7.83×10^2	7.95×10^2	8.34×10^2	8.65×10^2
	Rank	4	6	2	1	3	5
f8	Mean	9.27×10^2	1.31×10^3	9.41×10^2	9.08×10^2	9.13×10^2	9.38×10^2
	Std	2.85×10^1	4.39×10^1	3.38×10^1	3.25×10^1	3.94×10^1	2.25×10^1
	Min	8.71×10^2	1.20×10^3	8.64×10^2	8.66×10^2	8.59×10^2	8.73×10^2
	Rank	3	6	5	1	2	4
f9	Mean	3.58×10^3	2.88×10^4	4.18×10^3	2.06×10^3	2.73×10^3	4.43×10^3
	Std	9.32×10^2	4.75×10^3	1.88×10^3	1.00×10^3	1.15×10^3	4.86×10^2
	Min	1.93×10^3	1.95×10^4	1.62×10^3	1.10×10^3	1.48×10^3	2.46×10^3
	Rank	3	6	5	1	2	4
f10	Mean	5.05×10^3	1.06×10^4	5.49×10^3	5.01×10^3	5.04×10^3	4.89×10^3
	Std	6.39×10^2	4.48×10^2	6.19×10^2	7.87×10^2	7.72×10^2	4.84×10^2
	Min	3.74×10^3	9.36×10^3	3.97×10^3	3.84×10^3	3.58×10^3	2.97×10^3
	Rank	3	6	5	4	2	1
f11	Mean	1.34×10^3	5.44×10^4	1.58×10^3	1.40×10^3	1.35×10^3	1.27×10^3
	Std	1.03×10^2	6.52×10^4	3.46×10^2	1.19×10^2	9.36×10^1	5.36×10^1
	Min	1.20×10^3	1.26×10^4	1.20×10^3	1.21×10^3	1.19×10^3	1.15×10^3
	Rank	3	6	4	5	2	1
f12	Mean	1.77×10^7	2.11×10^6	3.96×10^8	6.49×10^7	5.83×10^7	6.38×10^7
	Std	1.10×10^7	2.20×10^6	5.06×10^8	2.52×10^8	1.44×10^8	7.73×10^7
	Min	2.03×10^6	1.28×10^5	1.51×10^6	1.14×10^5	1.08×10^5	2.57×10^5
	Rank	3	1	6	5	2	4
f13	Mean	9.34×10^4	1.56×10^4	5.50×10^7	5.18×10^6	4.53×10^6	1.08×10^5
	Std	4.47×10^4	1.11×10^4	2.09×10^8	1.71×10^7	1.72×10^7	6.85×10^4
	Min	2.83×10^4	1.72×10^3	2.89×10^4	1.06×10^4	4.79×10^3	2.14×10^4
	Rank	2	1	6	5	4	3
f14	Mean	9.08×10^4	9.57×10^3	1.01×10^5	6.45×10^4	5.56×10^4	2.97×10^4
	Std	1.19×10^5	1.22×10^4	1.03×10^5	6.66×10^4	6.09×10^4	3.01×10^4
	Min	1.91×10^3	1.88×10^3	4.63×10^3	3.79×10^3	4.60×10^3	1.61×10^3
	Rank	5	1	6	4	3	2
f15	Mean	6.05×10^4	3.43×10^3	9.28×10^4	3.89×10^4	2.11×10^4	3.30×10^4
	Std	1.02×10^5	2.60×10^3	7.17×10^4	4.91×10^4	2.27×10^4	1.89×10^4
	Min	8.00×10^3	1.58×10^3	6.59×10^3	2.43×10^3	1.96×10^3	5.54×10^3
	Rank	6	1	5	4	2	3
f16	Mean	2.78×10^3	8.59×10^3	3.03×10^3	2.72×10^3	2.70×10^3	2.98×10^3
	Std	3.05×10^2	1.82×10^3	3.25×10^2	3.67×10^2	3.51×10^2	2.69×10^2
	Min	2.22×10^3	5.94×10^3	2.34×10^3	2.06×10^3	2.17×10^3	2.13×10^3
	Rank	4	6	5	3	2	1

Table 4. Cont.

		VPPSO	MPSO	OldMPSO	AGPSO	IPSO	VASPSO
f17	Mean	2.16×10^3	1.78×10^4	2.37×10^3	2.24×10^3	2.24×10^3	2.34×10^3
	Std	2.32×10^2	2.17×10^4	2.70×10^2	2.17×10^2	2.59×10^2	2.36×10^2
	Min	1.79×10^3	3.65×10^3	1.95×10^3	1.88×10^3	1.91×10^3	1.81×10^3
	Rank	1	6	5	2	4	3
f18	Mean	8.54×10^5	2.46×10^5	9.10×10^5	5.67×10^5	4.98×10^5	8.19×10^5
	Std	8.15×10^5	3.34×10^5	1.35×10^6	1.04×10^6	1.30×10^6	1.33×10^6
	Min	6.40×10^4	3.47×10^4	7.82×10^4	5.27×10^4	6.81×10^4	1.22×10^4
	Rank	4	1	6	2	4	3
f19	Mean	1.40×10^6	5.85×10^3	8.04×10^6	3.07×10^5	1.16×10^5	8.60×10^4
	Std	8.17×10^5	4.27×10^3	2.88×10^7	1.49×10^6	1.49×10^6	2.59×10^5
	Min	1.12×10^5	1.95×10^3	3.77×10^3	2.38×10^3	2.14×10^3	2.29×10^3
	Rank	5	1	6	4	3	2
f20	Mean	2.56×10^3	2.34×10^3	2.57×10^3	2.51×10^3	2.52×10^3	2.63×10^3
	Std	1.92×10^2	1.50×10^2	2.22×10^2	2.01×10^2	2.15×10^2	1.85×10^2
	Min	2.22×10^3	2.04×10^3	2.20×10^3	2.17×10^3	2.21×10^3	2.19×10^3
	Rank	4	1	6	2	4	3
f21	Mean	2.43×10^3	2.90×10^3	2.46×10^3	2.42×10^3	2.42×10^3	2.47×10^3
	Std	3.49×10^1	3.01×10^1	3.98×10^1	3.08×10^1	3.40×10^1	2.85×10^1
	Min	2.37×10^3	2.77×10^3	2.40×10^3	2.36×10^3	2.37×10^3	2.37×10^3
	Rank	4	5	6	1	3	2
f22	Mean	3.44×10^3	2.72×10^3	5.82×10^3	4.70×10^3	4.37×10^3	5.26×10^3
	Std	1.84×10^3	7.33×10^2	1.55×10^3	1.92×10^3	2.00×10^3	1.45×10^3
	Min	2.30×10^3	8.90×10^3	2.64×10^3	2.41×10^3	2.30×10^3	2.30×10^3
	Rank	3	2	6	5	4	1
f23	Mean	2.81×10^3	3.87×10^3	2.91×10^3	2.85×10^3	2.88×10^3	3.00×10^3
	Std	4.10×10^1	1.99×10^2	7.61×10^1	6.70×10^1	7.78×10^1	7.18×10^1
	Min	2.72×10^3	3.46×10^3	2.82×10^3	2.76×10^3	2.77×10^3	2.79×10^3
	Rank	1	6	5	2	3	4
f24	Mean	2.95×10^3	4.23×10^3	3.10×10^3	3.05×10^3	3.10×10^3	3.20×10^3
	Std	3.72×10^1	3.10×10^2	8.70×10^1	8.94×10^1	1.01×10^2	5.92×10^1
	Min	2.88×10^3	3.53×10^3	2.97×10^3	2.96×10^3	2.96×10^3	2.96×10^3
	Rank	1	6	4	3	5	2
f25	Mean	2.93×10^3	2.96×10^3	3.12×10^3	2.94×10^3	2.92×10^3	2.95×10^3
	Std	2.56×10^1	2.76×10^1	1.73×10^2	6.88×10^1	5.59×10^1	2.06×10^1
	Min	2.89×10^3	2.89×10^3	2.88×10^3	2.89×10^3	2.88×10^3	2.88×10^3
	Rank	2	6	5	4	3	1
f26	Mean	4.91×10^3	4.41×10^3	6.00×10^3	4.97×10^3	5.23×10^3	6.34×10^3
	Std	1.29×10^3	1.11×10^3	7.65×10^2	7.67×10^2	9.54×10^2	1.03×10^3
	Min	2.80×10^3	2.81×10^3	4.04×10^3	2.80×10^3	2.80×10^3	2.80×10^3
	Rank	2	3	6	1	4	5
f27	Mean	3.29×10^3	3.25×10^3	3.29×10^3	3.27×10^3	3.27×10^3	3.20×10^3
	Std	3.84×10^1	2.18×10^1	7.01×10^1	4.98×10^1	5.01×10^1	2.01×10^{-4}
	Min	3.22×10^3	3.22×10^3	3.22×10^3	3.22×10^3	3.22×10^3	3.20×10^3
	Rank	3	2	5	4	6	1
f28	Mean	3.29×10^3	3.35×10^3	3.94×10^3	3.41×10^3	3.37×10^3	3.32×10^3
	Std	3.11×10^1	5.15×10^1	6.90×10^2	4.54×10^2	1.71×10^2	4.06×10^1
	Min	3.22×10^3	3.27×10^3	3.27×10^3	3.23×10^3	3.21×10^3	3.21×10^3
	Rank	2	4	6	5	3	1
f29	Mean	4.25×10^3	3.70×10^3	4.12×10^3	3.92×10^3	3.96×10^3	4.00×10^3
	Std	2.69×10^2	1.74×10^2	2.90×10^2	2.40×10^2	2.52×10^2	2.75×10^2
	Min	3.75×10^3	3.35×10^3	3.61×10^3	3.54×10^3	3.57×10^3	3.33×10^3
	Rank	6	1	5	2	4	3
f30	Mean	6.82×10^6	2.13×10^4	3.26×10^6	4.81×10^5	3.79×10^5	2.52×10^6
	Std	4.60×10^6	3.29×10^4	5.62×10^6	1.31×10^6	1.34×10^6	3.17×10^6
	Min	1.59×10^5	6.62×10^3	3.56×10^4	7.46×10^3	8.29×10^3	1.81×10^4
	Rank	6	1	5	2	3	4
	final rank	96	107	148	85	94	77

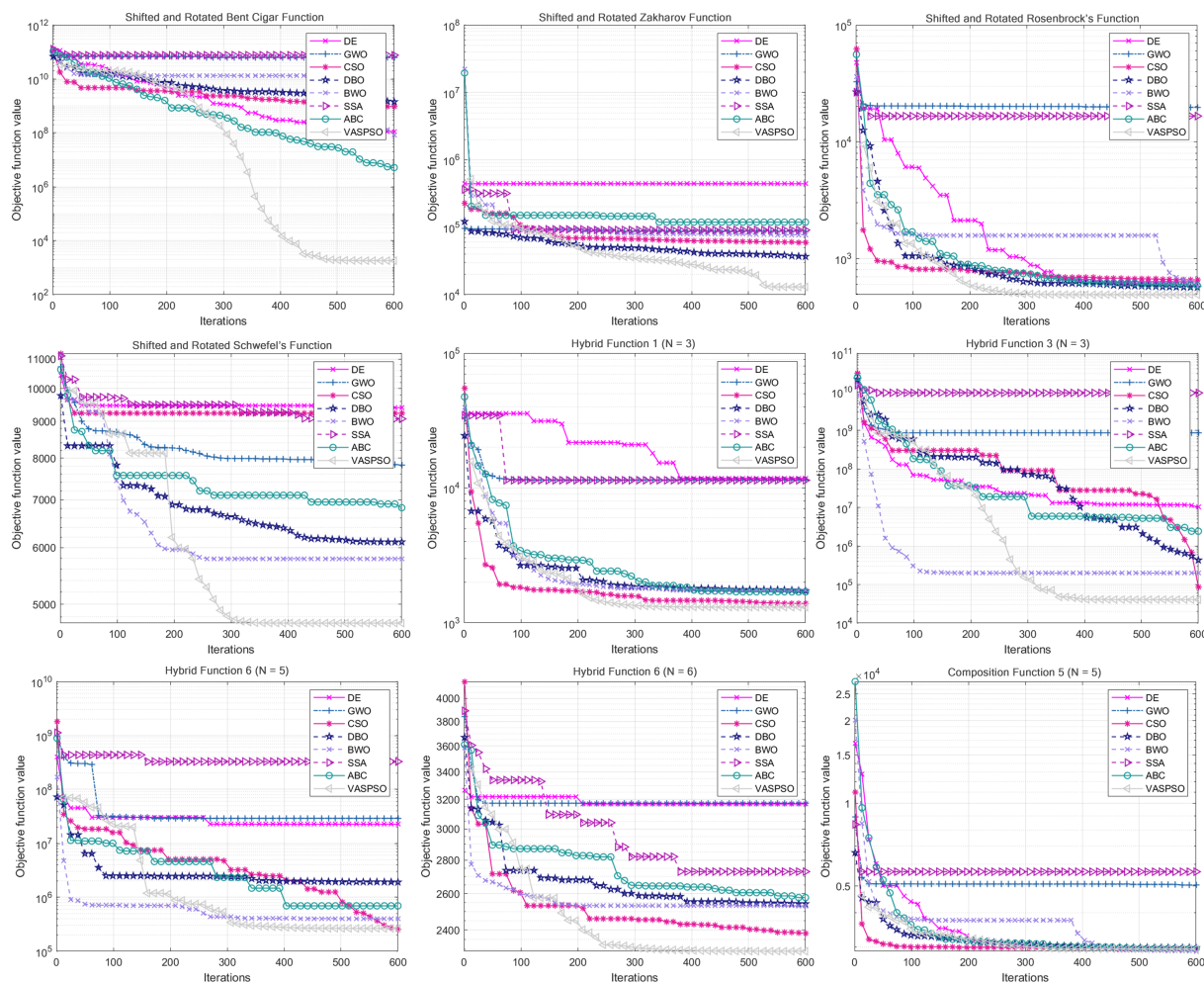


Figure 3. Convergence curve of algorithms for 7 evolutionary algorithms.

4.3. Analysis of Statistical Results

Statistical tests are typically required to analyze experimental results. The Friedman test, a non-parametric statistical method, is employed to assess whether significant differences exist in the median values of multiple paired samples across three or more related groups [53]. In the execution of the Friedman test, observations within each group are first ranked. Subsequently, the average ranks for each sample are calculated. The Friedman test statistic is then computed in a manner akin to that used in the analysis of variance, focusing on the sum of rank differences. This test is particularly suited for experimental designs involving multiple related samples, such as those evaluating the efficacy of different algorithms or treatment methods at various observation points. The progressive significance values from the Friedman test, as presented in Table 5, clearly demonstrate significant differences among the 12 algorithms analyzed.

The Wilcoxon signed-rank test is a non-parametric statistical method used to determine if there are significant differences between the median values of two related groups of samples. It is applicable in situations where the data do not follow a normal distribution and cannot be paired [54]. When conducting the Wilcoxon signed-rank test, the first step is to calculate the difference between each pair of related samples, then take the absolute value of these differences, and rank them in order of magnitude, assigning corresponding ranks. Next, the signs of the differences are disregarded, retaining only the ranks. Finally, the presence of significant differences between the median values of the two groups of samples is determined based on the sum of ranks.

Table 5. Comparison of algorithms based on the Friedman test.

Algorithm	Average Rank	Rank
BWO	11.82758621	12
SSA	11.62068966	11
ABC	8.379310345	10
OldMPSO	8.310344828	9
MPSO	7.137931034	8
CSO	5.75862069	7
DE	5.413793103	6
GWO	5.379310345	5
AGPSO	5.137931034	4
VPPSO	4.896551724	3
IPSO	4.482758621	2
VASPSO	4.103448276	1
<i>p</i> -value		1.4106×10^{-26}

In the Wilcoxon signed-rank test, the primary statistical measures of interest are the rank sum and the test statistic Z-value. Hypothesis testing is conducted by comparing the Z-value with a critical value or the p-value. If the Z-value is significantly greater than the critical value, or the p-value is less than the significance level, the null hypothesis can be rejected, indicating that there is a significant difference between the median values of the two sample groups. Considering a significance level of 0.05, and based on the statistical results presented in Table 6, it can be seen that the VASPSO algorithm outperforms the other algorithms.

Table 6. Comparison of algorithms based on the Wilcoxon signed-rank test.

Compared Algorithm	Number of Cases (−)	Number of Cases (+)	Z (Effect Size)	<i>p</i>
VASPSO - VPPSO	18	11	−1.175	0.2400685
VASPSO - GWO	16	13	−1.278	0.2012585
VASPSO - MPSO	18	11	−2.589	0.0096242
VASPSO - CSO	19	10	−1.941	0.052222
VASPSO - OldMPSO	26	3	−4.032	5.532×10^{-5}
VASPSO - DBO	25	4	−4.27	1.959×10^{-5}
VASPSO - IPSO	18	11	−0.782	0.4343045
VASPSO - BWO	29	0	−4.711	2.465×10^{-6}
VASPSO - AGPSO	20	9	−2.019	0.0434563
VASPSO - SSA	29	0	−4.711	2.461×10^{-6}
VASPSO - DE	17	12	−1.453	0.146127
VASPSO - ABC	23	6	−3.823	0.0001317

5. Conclusions

This study aims to mitigate several prevalent issues associated with the particle swarm optimization algorithm, such as premature convergence and the imbalance between global exploration and local exploitation, through the introduction of an enhanced version known as VASPSO. In our approach, we incorporate velocity pausing and a terminal replacement mechanism, which are specifically designed to prevent premature convergence. Additionally, VASPSO utilizes time-varying inertia coefficients, the concept of symmetric cooperative swarms, and adaptive strategies for modulating the search step length, which collectively contribute to a more balanced optimization process.

To rigorously evaluate the effectiveness of VASPSO in addressing complex optimization challenges, we conducted a series of comparative experiments using the CEC2017 benchmark. The results from these experiments suggest that VASPSO not only improves upon the performance of many existing PSO variants but also shows promising capabilities when compared with other well-regarded swarm intelligence algorithms.

However, VASPSO still has some limitations:

1. While VASPSO has shown promising results in experimental settings, the theoretical underpinnings and formal proofs supporting the VASPSO algorithm are still

somewhat limited. Further theoretical analysis is crucial to fully understand the mechanisms and principles driving its performance;

2. The classification of particles within the VASPSO algorithm currently employs a rigid structure that might oversimplify the inherent complexities of diverse optimization problems. This method of division may not adequately capture the nuanced characteristics of different scenarios, potentially limiting the algorithm's adaptability and effectiveness;
3. Certain parameters within the algorithm may require adjustments to effectively respond to variations in the problem's characteristics. A static parameter setting might not always align optimally with the dynamic nature of different optimization challenges;
4. VASPSO performs poorly on some benchmark functions (such as f_5 , f_6). Despite achieving good results on some optimization problems, the algorithm may exhibit relatively poorer performance on certain specific benchmark functions.

Future work will unfold in the following aspects:

1. Applying the VASPSO algorithm to real-world problems offers a significant opportunity to test and enhance its utility. For instance, in the field of engineering optimization, VASPSO could be particularly beneficial for complex tasks such as robot path planning and image processing. Integrating VASPSO with these practical applications not only allows for a robust evaluation of the algorithm's performance but also showcases its potential to provide effective and efficient solutions;
2. Integrating VASPSO with other optimization algorithms could significantly enhance its performance and robustness. By combining the strengths of multiple algorithms, a more effective balance between global exploration and local exploitation can be achieved. This hybrid approach not only improves the overall search capability of VASPSO but also enhances its convergence performance;
3. Combining VASPSO with neural networks can create a formidable optimization framework that leverages the strengths of both methodologies. By integrating the robust global search capabilities of VASPSO with the adaptive learning abilities of neural networks, this hybrid approach can facilitate more efficient parameter optimization and model training. This synergy enhances the algorithm's effectiveness in navigating complex parameter spaces and accelerates the convergence towards optimal solutions.

Author Contributions: Conceptualization, K.T.; Methodology, C.M.; Software, C.M.; Writing—original draft preparation, C.M.; Supervision, K.T.; Project administration, K.T.; Writing—review and editing, K.T.; Visualization, C.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no funding.

Data Availability Statement: The data that support the findings of this study are available from the corresponding author, Meng, C.-J., upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Mokeddem, D. A new improved salp swarm algorithm using logarithmic spiral mechanism enhanced with chaos for global optimization. *Evol. Intell.* **2022**, *15*, 1745–1775. [\[CrossRef\]](#)
2. Parsopoulos, K.E.; Vrahatis, M.N. Recent approaches to global optimization problems through particle swarm optimization. *Nat. Comput.* **2002**, *1*, 235–306. [\[CrossRef\]](#)
3. Kennedy, J.; Eberhart, R. Particle swarm optimization. In Proceedings of the ICNN'95-International Conference on Neural Networks, Perth, WA, Australia, 27 November–1 December 1995; IEEE: New York, NY, USA, 1995; Volume 4, pp. 1942–1948.
4. Wang, D.; Tan, D.; Liu, L. Particle swarm optimization algorithm: An overview. *Soft Comput.* **2018**, *22*, 387–408. [\[CrossRef\]](#)
5. Kiranyaz, S.; Ince, T.; Gabbouj, M.; Kiranyaz, S.; Ince, T.; Gabbouj, M. Particle swarm optimization. In *Multidimensional Particle Swarm Optimization for Machine Learning and Pattern Recognition*; Springer: Berlin/Heidelberg, Germany, 2014; pp. 45–82.
6. Shi, Y.; Eberhart, R.C. *Parameter Selection in Particle Swarm Optimization*; Springer: Berlin/Heidelberg, Germany, 1998.
7. Eberhart, R.; Shi, Y. Tracking and optimizing dynamic systems with particle swarms. In Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546), Seoul, Republic of Korea, 27–30 May 2001; Volume 1, pp. 94–100.

8. Chatterjee, A.; Siarry, P. Nonlinear inertia weight variation for dynamic adaptation in particle swarm optimization. *Comput. Oper. Res.* **2006**, *33*, 859–871. [\[CrossRef\]](#)
9. Feng, Y.; Teng, G.F.; Wang, A.X.; Yao, Y.M. Chaotic Inertia Weight in Particle Swarm Optimization. In Proceedings of the International Conference on Innovative Computing, Kumamoto, Japan, 5–7 September 2007.
10. Fan, S.K.S.; Chiu, Y.Y. A decreasing inertia weight particle swarm optimizer. *Eng. Optim.* **2007**, *39*, 203–228. [\[CrossRef\]](#)
11. Tang, Y.; Wang, Z.; Fang, J. Feedback learning particle swarm optimization. *Appl. Soft Comput.* **2011**, *11*, 4713–4725. [\[CrossRef\]](#)
12. Agrawal, A.; Tripathi, S. Particle Swarm Optimization with Probabilistic Inertia Weight. In *Harmony Search and Nature Inspired Optimization Algorithms*; Yadav, N., Yadav, A., Bansal, J.C., Deep, K., Kim, J.H., Eds.; Springer: Singapore, 2019; pp. 239–248.
13. Prastyo, P.H.; Hidayat, R.; Ardiyanto, I. Enhancing sentiment classification performance using hybrid query expansion ranking and binary particle swarm optimization with adaptive inertia weights. *ICT Express* **2022**, *8*, 189–197. [\[CrossRef\]](#)
14. Singh, A.; Sharma, A.; Rajput, S.; Bose, A.; Hu, X. An investigation on hybrid particle swarm optimization algorithms for parameter optimization of PV cells. *Electronics* **2022**, *11*, 909. [\[CrossRef\]](#)
15. Ratnaweera, A.; Halgamuge, S.; Watson, H. Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients. *IEEE Trans. Evol. Comput.* **2004**, *8*, 240–255. [\[CrossRef\]](#)
16. Beheshti, Z. A novel x-shaped binary particle swarm optimization. *Soft Comput.* **2021**, *25*, 3013–3042. [\[CrossRef\]](#)
17. Dixit, A.; Mani, A.; Bansal, R. An adaptive mutation strategy for differential evolution algorithm based on particle swarm optimization. *Evol. Intell.* **2022**, *15*, 1571–1585. [\[CrossRef\]](#)
18. Liu, W.; Wang, Z.; Zeng, N.; Yuan, Y.; Alsaadi, F.E.; Liu, X. A novel random particle swarm optimizer. *Int. J. Mach. Learn. Cybern.* **2021**, *12*, 529–540. [\[CrossRef\]](#)
19. Hu, X.; Eberhart, R. Multiobjective optimization using dynamic neighborhood particle swarm optimization. In Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No.02TH8600), Honolulu, HI, USA, 12–17 May 2002; Volume 2, pp. 1677–1681.
20. Liang, J.; Suganthan, P. Dynamic multi-swarm particle swarm optimizer. In Proceedings of the 2005 IEEE Swarm Intelligence Symposium, 2005. SIS 2005, Pasadena, CA, USA, 8–12 June 2005; pp. 124–129.
21. Varna, F.T.; Husbands, P. HIDMS-PSO Algorithm with an Adaptive Topological Structure. In Proceedings of the 2021 IEEE Symposium Series on Computational Intelligence (SSCI), Orlando, FL, USA, 5–7 December 2021; pp. 1–8.
22. Yang, Q.; Bian, Y.W.; Gao, X.D.; Xu, D.D.; Lu, Z.Y.; Jeon, S.W.; Zhang, J. Stochastic triad topology based particle swarm optimization for global numerical optimization. *Mathematics* **2022**, *10*, 1032. [\[CrossRef\]](#)
23. Potu, N.; Jatoth, C.; Parvataneni, P. Optimizing resource scheduling based on extended particle swarm optimization in fog computing environments. *Concurr. Comput. Pract. Exp.* **2021**, *33*, e6163. [\[CrossRef\]](#)
24. Liu, Z.; Nishi, T. Strategy Dynamics Particle Swarm Optimizer. *Inf. Sci.* **2022**, *582*, 665–703. [\[CrossRef\]](#)
25. Janakiraman, S.; Priya, M.D. Hybrid grey wolf and improved particle swarm optimization with adaptive inertial weight-based multi-dimensional learning strategy for load balancing in cloud environments. *Sustain. Comput. Inform. Syst.* **2023**, *38*, 100875. [\[CrossRef\]](#)
26. Molaei, S.; Moazen, H.; Najjar-Ghabel, S.; Farzinvas, L. Particle swarm optimization with an enhanced learning strategy and crossover operator. *Knowl.-Based Syst.* **2021**, *215*, 106768. [\[CrossRef\]](#)
27. Koh, W.S.; Lim, W.H.; Ang, K.M.; Isa, N.A.M.; Tiang, S.S.; Ang, C.K.; Solihin, M.I. Multi-objective particle swarm optimization with alternate learning strategies. In *Recent Trends in Mechatronics towards Industry 4.0: Selected Articles from iM3F 2020, Malaysia*; Springer: Singapore, 2022; pp. 15–25.
28. Wang, H.; Liang, M.; Sun, C.; Zhang, G.; Xie, L. Multiple-strategy learning particle swarm optimization for large-scale optimization problems. *Complex Intell. Syst.* **2021**, *7*, 1–16. [\[CrossRef\]](#)
29. Wang, R.; Hao, K.; Chen, L.; Wang, T.; Jiang, C. A novel hybrid particle swarm optimization using adaptive strategy. *Inf. Sci.* **2021**, *579*, 231–250. [\[CrossRef\]](#)
30. Angeline, P.J. Using selection to improve particle swarm optimization. In Proceedings of the Evolutionary Computation Proceedings, 1998. IEEE World Congress on Computational Intelligence, Anchorage, AK, USA, 4–9 May 1998.
31. Løvbjerg, M.; Rasmussen, T.K.; Krink, T. Hybrid Particle Swarm Optimiser with breeding and subpopulations. In Proceedings of the Genetic and Evolutionary Computation Conference, San Francisco, CA, USA, 7–11 July 2001.
32. Chen, Y.P.; Peng, W.C.; Jian, M.C. Particle Swarm Optimization With Recombination and Dynamic Linkage Discovery. *IEEE Trans. Syst. Man Cybern. Part B (Cybernetics)* **2007**, *37*, 1460–1470. [\[CrossRef\]](#)
33. Andrews, P. An Investigation into Mutation Operators for Particle Swarm Optimization. In Proceedings of the 2006 IEEE International Conference on Evolutionary Computation, Vancouver, BC, Canada, 16–21 July 2006; pp. 1044–1051.
34. Yu, Z.; Si, Z.; Li, X.; Wang, D.; Song, H. A Novel Hybrid Particle Swarm Optimization Algorithm for Path Planning of UAVs. *IEEE Internet Things J.* **2022**, *9*, 22547–22558. [\[CrossRef\]](#)
35. Zhang, X.; Lin, Q.; Mao, W.; Liu, S.; Liu, G. Hybrid Particle Swarm and Grey Wolf Optimizer and its application to clustering optimization. *Appl. Soft Comput.* **2020**, *101*, 107061. [\[CrossRef\]](#)
36. Koessler, E.; Almomani, A. Hybrid particle swarm optimization and pattern search algorithm. *Optim. Eng.* **2021**, *22*, 1539–1555. [\[CrossRef\]](#)
37. Adamu, A.; Abdullahi, M.; Junaidu, S.B.; Hassan, I.H. An hybrid particle swarm optimization with crow search algorithm for feature selection. *Mach. Learn. Appl.* **2021**, *6*, 100108. [\[CrossRef\]](#)

38. Pozna, C.; Precup, R.E.; Horváth, E.; Petriu, E.M. Hybrid Particle Filter–Particle Swarm Optimization Algorithm and Application to Fuzzy Controlled Servo Systems. *IEEE Trans. Fuzzy Syst.* **2022**, *30*, 4286–4297. [[CrossRef](#)]
39. He, W.; Qi, X.; Liu, L. A novel hybrid particle swarm optimization for multi-UAV cooperate path planning. *Appl. Intell.* **2021**, *51*, 7350–7364. [[CrossRef](#)]
40. Shami, T.M.; Mirjalili, S.; Al-Eryani, Y.; Daoudi, K.; Izadi, S.; Abualigah, L. Velocity pausing particle swarm optimization: A novel variant for global optimization. *Neural Comput. Appl.* **2023**, *35*, 9193–9223. [[CrossRef](#)]
41. Awad, N.; Ali, M.; Liang, J.; Qu, B.; Suganthan, P. Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective bound constrained real-parameter numerical optimization. In *Technical Report*; Nanyang Technological University Singapore: Singapore, 2016; pp. 1–34.
42. Liu, H.; Zhang, X.W.; Tu, L.P. A modified particle swarm optimization using adaptive strategy. *Expert Syst. Appl.* **2020**, *152*, 113353. [[CrossRef](#)]
43. Bao, G.Q.; Mao, K.F. Particle swarm optimization algorithm with asymmetric time varying acceleration coefficients. In Proceedings of the IEEE International Conference on Robotics and Biomimetics, Guilin, China, 18–22 December 2009; pp. 2134–2139.
44. Mirjalili, S.; Lewis, A.; Sadiq, A.S. Autonomous Particles Groups for Particle Swarm Optimization. *Arab. J. Sci. Eng.* **2014**, *39*, 4683–4697. [[CrossRef](#)]
45. Cui, Z.; Zeng, J.; Yin, Y. An improved PSO with time-varying accelerator coefficients. In Proceedings of the Eighth International Conference on Intelligent Systems Design and Applications, Kaohsiung, Taiwan, 26–28 November 2008; pp. 638–643.
46. Storn, R.; Price, K. Differential Evolution—A Simple and Efficient Heuristic for global Optimization over Continuous Spaces. *J. Glob. Optim.* **1997**, *11*, 341–359. [[CrossRef](#)]
47. Mirjalili, S.; Mirjalili, S.M.; Lewis, A. Grey Wolf Optimizer. *Adv. Eng. Softw.* **2014**, *69*, 46–61. [[CrossRef](#)]
48. Meng, X.; Liu, Y.; Gao, X.; Zhang, H. A New Bio-inspired Algorithm: Chicken Swarm Optimization. In Proceedings of the International Conference in Swarm Intelligence, Hefei, China, 17–20 October 2014.
49. Xue, J.; Shen, B. Dung beetle optimizer: A new meta-heuristic algorithm for global optimization. *J. Supercomput.* **2023**, *79*, 7305–7336. [[CrossRef](#)]
50. Zhong, C.; Li, G.; Meng, Z. Beluga whale optimization: A novel nature-inspired metaheuristic algorithm. *Knowl.-Based Syst.* **2022**, *251*, 109215.
51. Xue, J.; Shen, B. A novel swarm intelligence optimization approach: Sparrow search algorithm. *Syst. Sci. Control Eng. Open Access J.* **2020**, *8*, 22–34. [[CrossRef](#)]
52. Karaboga, D.; Basturk, B. A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *J. Glob. Optim.* **2007**, *39*, 459–471. [[CrossRef](#)]
53. Sheldon, M.R.; Fillyaw, M.J.; Thompson, W.D. The use and interpretation of the Friedman test in the analysis of ordinal-scale data in repeated measures designs. *Physiother. Res. Int. J. Res. Clin. Phys. Ther.* **1996**, *14*, 221–228. [[CrossRef](#)]
54. Derrac, J.; García, S.; Molina, D.; Herrera, F. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. *Swarm Evol. Comput.* **2011**, *1*, 3–18. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.